

Adaptive Merkle Trie Versioning for Efficient Structured Data Management

Atharva Sheersh Pandey¹, Adhyan Jain¹, Poornima Nedunchezian²
atharva.sheersh2024@vitstudent.ac.in; adhyan.jain2024@vitstudent.ac.in;
poornima.n@vit.ac.in

School of Computer Science and Engineering
VIT Vellore

Abstract: Revon is a Python prototype for durable, linear-history key/value versioning using a fixed-depth Merkle hash trie. Its hybrid variant chooses between changeset aggregation and hash-pruned comparison; trie geometry does not adapt. We make forced-Merkle Revon-M the primary ablation because both variants retain the same persistent representation and changesets. On eight synthetic workloads, five paired diff-latency intervals favour Revon-H and three include parity. A separate three-seed study also leaves the 50-commit result inconclusive. Public NYC TLC records extend evaluation to one million records with generated corrections; Revon-H has no consistent advantage when both variants choose Merkle comparison. A supplemental study tests five thresholds on disjoint calibration and validation workloads, with seven measured cached queries per configuration. Calibration selects 16,384 operations, while the original workflow results retain their frozen 4,096 threshold. No forced-path crossover appears through 32,768 operations in the main supplement; a separate long-history diagnostic investigates larger counts. Revon-H and Revon-M have nearly equal repository footprints, whereas both use more storage than Dolt in the measured workflows. All measurements use one physical computer, including WSL2. The evidence supports workload-dependent differencing choices, not general efficiency or production-system superiority.

Keywords: data versioning, Merkle hash trie, hybrid differencing, content-addressed storage, benchmark methodology

I. INTRODUCTION

Structured-data version control has to do more than archive files. It must preserve exact versions, apply sparse updates without rebuilding the whole state, restore earlier versions, and compare states efficiently. Git offers a useful starting point: immutable objects named by their content, linked through parent commits [1]. Its objects are organized around files and directories, however. Record stores have a different shape: keys may remain stable while update density changes, and two versions may differ by a few edits or by a long commit history.

Revon asks whether a fixed-depth Merkle hash trie can combine incremental structural sharing with an operation index. Revon-H adapts only its diff path: it selects log aggregation for eligible intervals within an operation threshold and Merkle comparison otherwise. The trie geometry remains fixed. SQLite supplies atomic, durable storage. The title's efficiency claim is restricted to the measured operations and workloads; it does not assert that all structured-data management becomes cheaper.

We use Dolt as the production comparator because it combines SQL, a Git-style commit graph, and content-addressed Prolly Trees [2], [3]. Revon is a smaller research prototype, not a feature-for-feature replacement for Dolt. Our results compare Revon's in-process Python API with a Dolt CLI workflow; they do not isolate the performance of either tree structure.

A. Research Questions

RQ1: How do Revon-H's latency and storage compare with Snapshot, Log-only, and Dolt across dataset sizes, mutation densities, and update localities?

RQ2: Does choosing between log and Merkle diff improve latency over forced-Merkle Revon-M while preserving correctness?

RQ3: How do branching factor and trie depth affect latency, storage, and the amount of work examined?

B. Contributions

We implement a canonical SHA-256-addressed, fixed-depth trie with incremental rewriting of paths affected by multi-key updates.

We design a hybrid diff policy that combines addressed changesets with hash-pruned tree comparison.

We build an atomic SQLite repository for objects, commits, aliases, measurements, and HEAD, and verify object hashes after reopening it.

We compare five systems on shared deterministic workloads, using common diff and checkout outputs. We report Dolt's two initial-import workflows separately, randomize and balance execution order, sample process-tree RSS externally, retain raw evidence, and audit its integrity.

We measure how branching factor and depth affect the fixed trie, rather than assuming that $b=8$ and $d=4$ are universally best.

II. RELATED WORK

A. Content Addressing and Persistent Structures

Merkle’s authenticated-tree construction showed that a compact root digest can commit to a larger collection [4]. Git and IPFS apply content addressing to software history and networked content [1], [6]. Persistent-data-structure theory formalized path copying and access to earlier states [5]. Later persistent dictionaries explore ordered, disk-oriented alternatives: stratified B-trees [16] and fully persistent B-trees [19]. Revon instead routes hashed keys through a fixed-depth trie, trading ordered range access for hash-pruned comparison. Scientific array systems such as SciDB and TimeArr preserve versions using array-specific storage layouts and deltas [17], [18]. TardisDB integrates version access with a main-memory database, while Saquib et al. study replicated versioned structures across wide-area systems [20], [21]. These systems illuminate adjacent design choices, but they do not compare Revon’s log and Merkle diff paths over the same history.

Merkle Search Trees combine search-tree ordering with Merkle authentication to reconcile state [7]. Revon also prunes comparisons by hash, but uses fixed-depth routing and a centralized linear history rather than a replicated CRDT. LBFS uses content-defined chunking for redundancy detection [8]; Dolt and Noms use related boundary strategies. Ordered indexes preserve range operations, while content-defined boundaries can improve deduplication when data shifts. Revon’s current benchmark measures neither capability.

B. Structured Dataset Versioning

Dataset-versioning research covers collaboration, storage layout, and retrieval. DataHub studies collaborative dataset

history at scale [9]. Decibel brings branching into a relational engine and compares version-first, tuple-first, and hybrid layouts [10]. OrpheusDB adds versioning to a conventional DBMS and optimizes version retrieval through partitioning [11]. Bhattacharjee et al. formalize the storage-versus-reconstruction trade-off [12], while Delta Lake uses transaction logs and checkpoints for ACID tables [13]. More recent work studies how to choose materialized versions and deltas over a version graph, subject to storage and retrieval costs [26]. A FAIR-compliant proposal focuses on identifying and describing dataset releases consistently [27]; it complements storage research but does not provide a diff algorithm. Revon addresses a narrower question with a key/value prototype and does not solve these broader dataset-management problems.

C. System Design Gap

Noms stores structured values in a Merkle DAG and supports diff and synchronization [14]. ForkBase combines content addressing with fork and deduplication semantics [15]. Dolt’s content-addressed Prolly Trees combine ordered B-tree-like indexing, structural sharing, and native diff [2], [3]. Git4Data is another nearby direction: it presents SQL-level snapshot, branch, diff, and merge operations over a relational engine [29]. The available Git4Data manuscript is an arXiv preprint, so we do not treat its results here as peer-reviewed or independently reproduced evidence. Revon has a narrower scope: it is a single-writer, linear-history key/value prototype with fixed-depth hash routing, not a SQL database or branching system.

Table I compares the related systems and the scope addressed by Revon.

TABLE I. Related work and the scope of Revon

System or study	Representation or focus	Relevance to Revon
Persistent DS [5]	Path copying	Formal structural-sharing basis
Git [1]	Object DAG	Identity and commit ancestry; file-oriented
IPFS [6]	Merkle DAG	General content-addressed versioning
MST [7]	Merkle search tree	Authenticated ordered reconciliation
DataHub [9]	Version graph	Collaborative dataset motivation
Decibel [10]	Relational layouts	Native database branching
OrpheusDB [11]	Partitioned versions	Storage/retrieval optimization
Delta Lake [13]	Log/checkpoints	ACID time travel at data-lake scale
Noms [14]	Merkle DAG	Structured content addressing
ForkBase [15]	Forkable CAS	Fork and deduplication semantics
DoltHub [2,3]	Prolly Trees	Mature production comparator

D. Design Implications and Research Positioning

Versioning systems face three recurring costs: retaining history, creating a version, and later reconstructing or comparing versions. Snapshots simplify retrieval by copying unchanged records. Logs defer work until replay or checkpointing. Path copying creates only changed paths and shares the rest [5], [12]. Decibel and OrpheusDB show that physical layout affects both version creation and retrieval [10], [11]; graph-based analyses make the same storage/retrieval trade-off explicit [26]. Reproducible-research work addresses a separate problem: preserving builds and artifacts for later inspection does not show that one storage algorithm is faster [24]. For that reason, Revon reports import, commit, checkout, diff, and storage measurements separately.

Different research areas also give “change” different meanings. Differential dataflow incrementally recomputes query results as timestamped inputs change [23]. Data-provenance research asks why and where output facts were derived [22], and DSLog stores and queries fine-grained array lineage [28]. Explain-Da-V uses data transformations to explain semantic dataset changes [25]. Baseline uses operation-based differencing that can account for refactorings and schema changes [30]. Revon reports a more limited result: which keys were added, removed, or changed in value between two states. It does not infer causes,

transformation lineage, query impact, or schema intent. These systems offer useful context, but their outputs and workloads cannot be compared directly with Revon’s key/value diff timings.

Revon’s contribution is a controlled comparison of two correct diff paths over the same durable history, selected using properties of the requested interval. Each Revon-H commit stores both a Merkle root and an addressed changeset, so the choice of diff path does not change version semantics. We hypothesize that log aggregation suits short, sparse intervals and that hash pruning may help after longer operation sequences. A crossover must be measured rather than inferred from the switching rule. Work such as Git4Data also shows that database-native version control remains an active area [29], which narrows our claim: Revon evaluates adaptive differencing for a fixed-depth hash trie; it is not a general-purpose data-versioning platform. Snapshot and Log-only remain representation baselines. The public-record extension covers one release with generated updates. It does not establish performance across diverse datasets, independent hardware, concurrent workloads, or production systems.

III. REVON DESIGN

Figure 1 summarizes the architecture and implemented research scope.

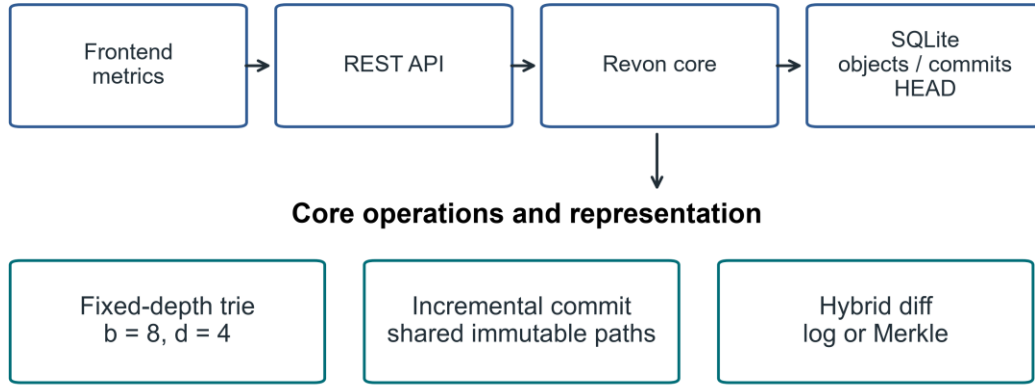


Fig. 1. Revon architecture and the implemented research scope. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

A. Addressed Object Model

Revon serializes trie nodes and changesets as canonical JSON and names each object by its SHA-256 digest. A commit digest covers the root, parent, changeset, message, and timestamp. Labels such as v1 and v2 are readable aliases only; they do not affect object identity. A durable HEAD reference identifies the current commit.

B. Fixed-Depth Merkle Hash Trie

The main experiment uses $b=8$ and $d=4$. At each level, Revon reads three bits from SHA-256(key), so twelve routing bits select among 4,096 possible leaf buckets. Each bucket retains the full key, allowing Revon to distinguish keys with the same routing prefix. With uniform hashes, expected bucket occupancy is N/b^d . We evaluate other b, d pairs separately rather than treating this default as optimal.

C. Incremental Hashing and Commit

Revon canonicalizes each mutation batch, routes the keys, and groups updates that share trie prefixes. It rewrites each affected leaf once, then rebuilds only the ancestor paths that contain changes. Unchanged child hashes are reused in the new internal nodes, which preserves structural sharing. A single SQLite transaction writes the new objects, changeset, commit metadata, version alias, measurements, and updated

HEAD. On reopen, Revon verifies the stored payload hashes and references.

D. Hybrid Diff

The log path walks ancestry and aggregates key operations; the Merkle path compares roots recursively, skips equal hashes, and examines entries under unequal paths. Revon-H validates both version identifiers, checks ancestry in either direction, and counts changeset operations. It chooses log aggregation when an ancestor path exists and the count is at most T ; otherwise it uses Merkle comparison. Reverse log requests swap old/new values, and a same-version request returns an empty diff. Forced log mode rejects unrelated versions. Unknown versions raise an error. Revon-M forces Merkle comparison, although its public version-pair API also checks ancestry. Both paths return the same sorted semantic diff. Commit and checkout are separate operations.

The original synthetic and public workflow matrices retain $T=4,096$. The supplemental calibration evaluates other thresholds without changing those archived results. The 4,096 possible leaf buckets arise from $b=8, d=4$ and are independent of the operation threshold.

Figure 2 shows the implemented version-pair decision, including eligibility and reverse handling.

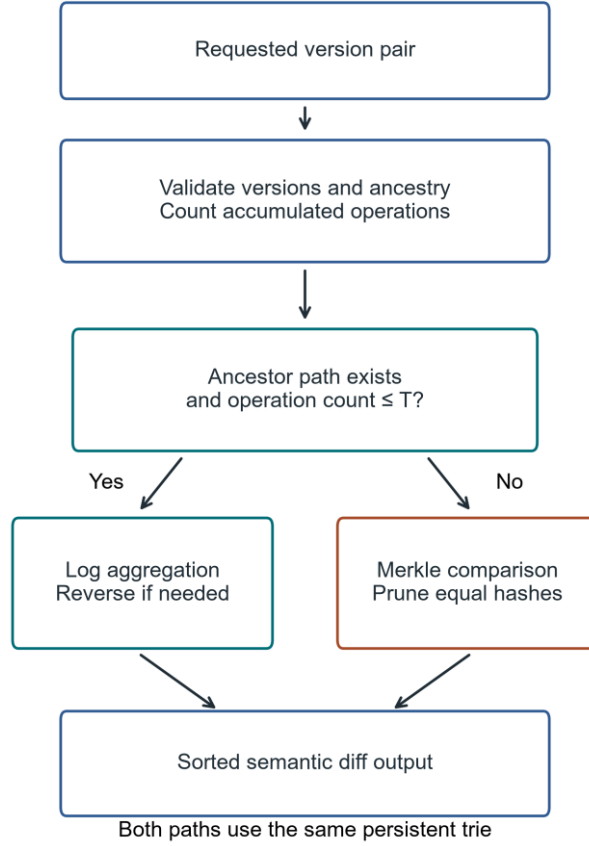


Fig. 2. Actual hybrid selection for a version pair; T denotes the operation threshold. Invalid versions raise an error before selection. Both paths share the persistent trie. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

IV. EXPERIMENTAL METHODOLOGY

A. Environment and evidence bundles

Table II separates the environments and measurement scopes of the retained evidence bundles.

TABLE II. Environment and the distinct evidence bundles

Bundle	Environment and configuration	Measurement scope
Primary synthetic	Windows 11; Python 3.14.3; Dolt 2.3.1; b8-d4, T4096	540 executions; 2 warm-ups and 7 measured trials per configuration
Robustness	Same Windows host; three seeds; 10k rows; H10/H50	Seed-cluster intervals, reported separately
WSL sensitivity	Same computer; Linux 6.6.87.2-microsoft-standard-WSL2; Python 3.14.4	Operating-environment comparison, not independent hardware
Public TLC	Same Windows host; Python 3.12.14; pyarrow 25.0.1; psutil 7.2.2; Dolt 2.3.1	325 attempts; 324 successful; 1 interrupted attempt retained; recovery outside block
Threshold supplement	Same Windows host; Python 3.14.3; 10k/100k rows; b8-d4	42 repositories; 2 warm-up and 7 measured queries per path/threshold
Physical host	Intel Core i7-1255U; 10 cores/12 logical CPUs; 15.64 GiB RAM; C: NTFS	CPU model/filesystem verified for this revision; historical manifests contain processor family
Independent Linux host	Not available to this revision	Independent replication remains uncompleted

These bundles share a physical host but differ in runtime, workloads and protocol. Results are not pooled across them. The public-data run started with 4.14 GiB available RAM; available memory was not controlled. The current CPU model and filesystem were inspected during revision, not retrospectively recorded for each historical trial. Recorded versions in each manifest remain authoritative.

B. Variants and synthetic workloads

We compare five systems on the same logical states and mutation batches: Snapshot stores a complete copy per

TABLE III. Synthetic workloads in the primary evaluation

Scenario	Rows	Commits	Changed keys	Update locality
Small sparse	1,000	10	10	spread
Medium sparse	10,000	10	10	spread
Medium dense	10,000	10	1,000	spread
Large sparse	100,000	10	100	spread
Application-key local	100,000	10	100	lexically adjacent keys
Range local	100,000	10	100	contiguous sorted-key intervals
Repeated key	100,000	10	100	repeated small key set
Hash-route local	100,000	10	100	shared 9-bit SHA-256 prefix

C. Timing correctness and uncertainty

We time initial import through the first durable state. For each trial, incremental-commit time is the median across ten commits. Diff and checkout timers include construction of a common output: diff returns sorted changed keys with old and new SHA-256 value hashes as canonical JSON, while checkout returns the full sorted key/value state as canonical UTF-8 JSON. Dolt’s diff time includes JSON parsing and normalization. We check both outputs against the workload oracle. Dolt’s SQL-path commit time includes SQL mutations, add, and commit; we retain commit hashes for historical checkout and create no version tags. Timed operations run without tracemalloc. During each isolated primary trial, an external parent process samples process-tree RSS every 10 ms, including Dolt child processes; sampling happens outside the timed operations. A separate Windows telemetry run records RSS, CPU, read/write bytes, and serial commit throughput in 405 rows (315 measured). Every row passed correctness checks and the schema-2 audit. These counters include setup and correctness work, not just timed operations. The WSL2 replication has incomplete I/O coverage and ran on the same physical host. Work-examined units differ by system and are not normalized.

The primary matrix uses two warm-ups and seven measured trials per configuration, for 120 warm-ups and 420 measured runs across calibration and evaluation. Of these, 280

version; Log-only stores an initial state and operation log; Revon-M forces Merkle diff; Revon-H selects its diff path adaptively; and Dolt 2.3.1 uses a keyed SQL table and native JSON diff. We measure Dolt’s initial import in two ways: batched SQL INSERT and CSV input through dolt table import -r. The CSV workflow includes serialization in its import time. Both import variants use the same Dolt engine for later operations, and we report their import results separately.

Table III defines the primary synthetic workload matrix.

measured runs belong to the five-system evaluation, including 56 SQL-path Dolt runs; another 56 measured runs use Dolt bulk import. We counterbalance and record model order, and run each system/trial in a clean worker. Each scenario uses one deterministic workload, which the internal audit regenerates and checks by digest. The primary run used Windows 11 build 10.0.26200-SP0 and Python 3.14.3 (run b7a4ec7b610742d9b827f43fd5092fe2). For each scenario and comparison, we divide latency measurements from matching trial numbers and bootstrap the seven paired ratios 20,000 times to obtain a percentile 95% interval. These intervals describe variation across runs on this fixed workload and host; they do not measure variation across datasets or machines. We label intervals that include parity as inconclusive. The 5% practical margin is an analyst-selected label, not an external standard.

The counterbalanced primary design pairs same-trial ratios within a scenario. Seven trials estimate repeated-run uncertainty on a fixed workload and host. The public-data ablation instead uses exploratory unpaired percentile bootstrap intervals for ratios of medians, with 20,000 resamples and seed 20261002. This avoids treating the isolated recovery as part of the original block. IQRs describe dispersion and are not confidence intervals. Repeated cached-query intervals in the threshold supplement do not estimate uncertainty across independent repository builds.

The prior calibration sampled through 4,096 operations and did not locate a crossover. Its 540 successful primary executions and all 127 Tukey outlier candidates are retained. New threshold results are supplemental measurements, not replacements for those trials.

D. Public dataset and scalability evaluation

We extend the synthetic evaluation with January 2024 yellow-taxi records from the New York City Taxi and Limousine Commission (NYC TLC) [31]. The checksum-pinned Parquet file contains 2,964,624 rows; a dispersed affine permutation selects nested subsets of 10,000, 100,000 and 1,000,000 rows using seed 20261001. We retain eleven fields: vendor, pickup/drop-off times, passenger count, distance, pickup/drop-off locations, payment type, fare, tip and total. Keys combine the release, file-checksum prefix and physical row ordinal, and therefore identify rows within this frozen file rather than trips across releases. Compact sorted-key UTF-8 JSON preserves values; nulls remain null. The million-row subset has 47,277 missing passenger counts and no serialization failures. Identical field values remain separate rows. The affine sample is not a uniformly random permutation.

Monthly trip files contain different trips, so we generate synthetic correction histories over these public records. Each commit samples distinct keys with a fixed seed, increases fare and total by 0.01, and preserves the other fields; keys may repeat across commits. Table IV specifies the separate scale, history and density sweeps. All systems receive identical states and histories. We retain $b=8$, $d=4$ and the 4,096-operation selector. Diff compares version 1 with version $H+1$. Import is separate from incremental version creation; each trial reports the median of H commit intervals.

TABLE IV. Public TLC dataset [31], January 2024; all histories are generated corrections

Parameter	Dataset and sweep specification
Dataset	NYC TLC yellow taxi, January 2024 [31]
Records	2,964,624 source rows; 1,000,000 retained; nested subsets of 10k, 100k and 1M
Fields	Vendor; pickup/drop-off times; passenger count; distance; pickup/drop-off locations; payment type; fare; tip; total
Key	Month + source-checksum prefix + physical row ordinal; identity within the frozen file
Histories	Generated fare/total corrections; 10 or 50 incremental commits; no observed release histories
Scale	$N=10k/100k/1M$; $H=10$; $C=10/100/1,000$ per commit (0.1%)
History	$N=100k$; $H=10/50$; $C=100$ per commit (0.1%)
Density	$N=100k$; $H=10$; $C=10/100/1,000$ per commit (0.01%/0.1%/1%)
Sampling	Seed 20261001; nested prefixes of a dispersed affine permutation; not a uniform random permutation

Canonical output materialization is included in diff and checkout. Every trial checks both endpoint states and semantic diff; the first measured trial for each system/configuration checks every historical version outside timing. Directory footprint is measured before close and compaction. These are whole-system workflow measurements.

The public-data run used Python 3.12.14 on the same Windows host, with 15.64 GiB visible RAM and 4.14 GiB available at launch. Available memory was not held constant. Each configuration scheduled 2 warm-ups and 7 measured trials. All configurations have 7 successful measured trials. Clean workers and counterbalanced order follow the existing protocol; pilot measurements are excluded. Each trial has a 900-second wall-clock limit including setup and verification. We report medians and linearly interpolated interquartile ranges. 324 of 325 retained executions completed successfully. The source checksum, workload digests, raw trials, environment metadata, verification hashes and reproduction commands accompany the new evidence; prior bundles are preserved.

The public run retains 72 warm-ups and 252 successful measured executions across 36 configurations. One additional measured attempt was interrupted. Recovery contributes one of the 252 successful measurements and ran outside the original counterbalanced block. The evidence includes 1,221 successful historical state-hash checks; pilots do not enter reported performance summaries.

Table IV summarizes the public dataset mapping and generated history sweeps.

E. Threshold calibration and held-out validation

Before measuring, we specify five candidates: 1,024, 2,048, 4,096, 8,192 and 16,384 operations. Three calibration profiles use 10k/H4/spread, 100k/H16/spread and 10k/H64/repeated-key workloads. Three validation profiles use distinct seeds and 100k/H4/repeated-key, 10k/H16/spread and 100k/H64/spread workloads. Each profile is tested at 512, 1,024, 2,048, 4,096, 8,192, 16,384 and 32,768 accumulated operations, with 32-byte synthetic values. Changes per commit equal accumulated operations divided by history length, so sizes, histories and densities vary; this is not a full factorial population study.

One durable repository is constructed per workload, then forced log, forced Merkle and all five selectors are measured in deterministically shuffled query blocks with two warm-ups and seven measured queries. Timers include `diff_versions` and canonical semantic output materialization. Correctness and separate decision probes are outside the `diff`

timer. The model caches stored objects in memory; these are repeated in-process queries, not independent repository trials or cold I/O measurements. Endpoint and midpoint states, reverse and identity diffs, integrity hashes, and reopen checks pass for all 42 repositories and 2,646 queries, including 2,058 measured queries.

Threshold selection minimizes the equal-weight mean of each calibration-case selector median divided by the faster forced-path median; an exact score tie selects the smaller threshold. The choice is recorded before validation starts. Validation profiles do not influence that choice. Both requested and actual selected paths are retained. A standalone ancestry/count probe has configuration medians of 0.0038-0.1186 ms in the main supplement. It measures decision work separately and is not subtracted from total latency as a causal estimate of selector overhead.

Figure 3 compares forced-path latency across the predefined calibration and validation profiles.

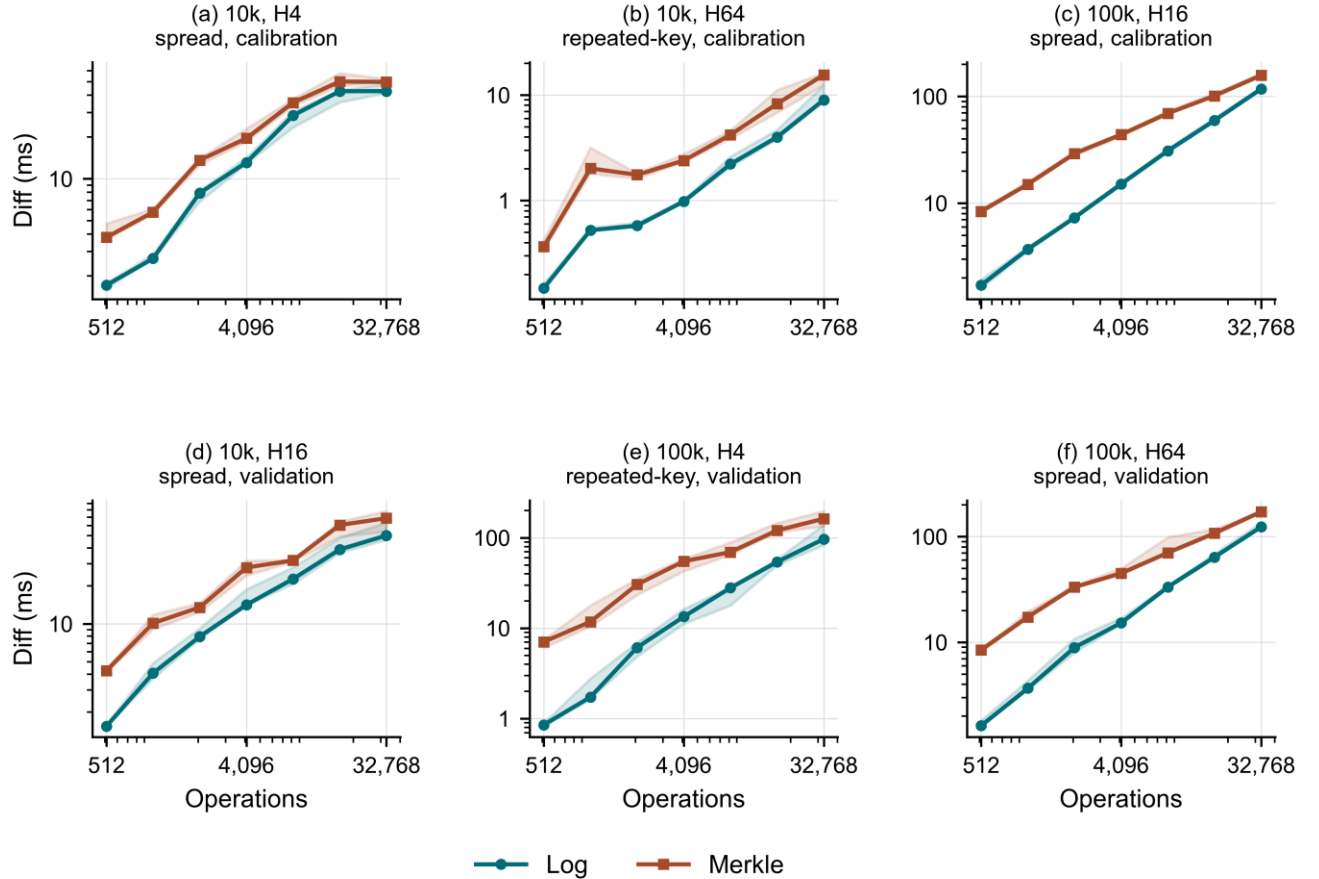


Fig. 3. Forced-path latency across calibration and held-out profiles; medians and IQRs from seven measured cached queries per configuration. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

Figure 4 shows how changing the threshold affects selector latency in held-out workloads.

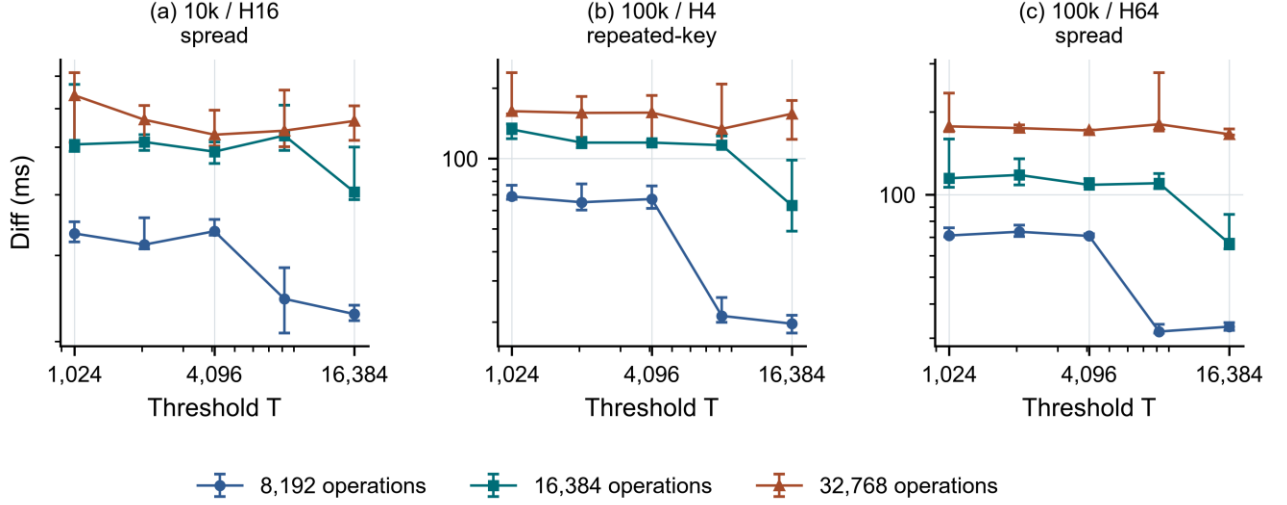


Fig. 4. Held-out selector latency versus switching threshold for 8,192, 16,384 and 32,768 accumulated operations; medians and IQRs. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

Table V compares the predefined threshold candidates using calibration and held-out scores.

TABLE V. Candidate scores; equal-weight mean latency relative to the faster forced-path median, lower is better

Threshold	Calibration score	Held-out score	Selection
1,024	1.669	1.903	Candidate
2,048	1.432	1.555	Candidate
4,096	1.234	1.297	Candidate
8,192	1.133	1.149	Candidate
16,384	1.062	1.032	Frozen choice

Calibration selects $T=16,384$, the largest tested candidate. Its held-out normalized score is 1.032, versus 1.297 for $T=4,096$. These scores are descriptive aggregates across the predefined cases, not population confidence intervals. Forced log remains faster at every largest sampled count through 32,768 in the main supplement. Consequently, the candidate winner is bounded by the search range and does not identify an optimal crossover. Per-case lowest-median candidates are retained in the supporting audit; held-out minima are diagnostic and never used to reselect T .

A separate exploratory extension investigates a 10k-row repeated-key workload with 256 commits and 8,192 through 131,072 operations. Its seed and five operation counts are fixed before extension measurements. The 315 queries, including 245 measured queries, pass the same checks. This extension is excluded from calibration and validation scores.

Figure 5 examines the separate long-history diagnostic without changing the calibrated choice.

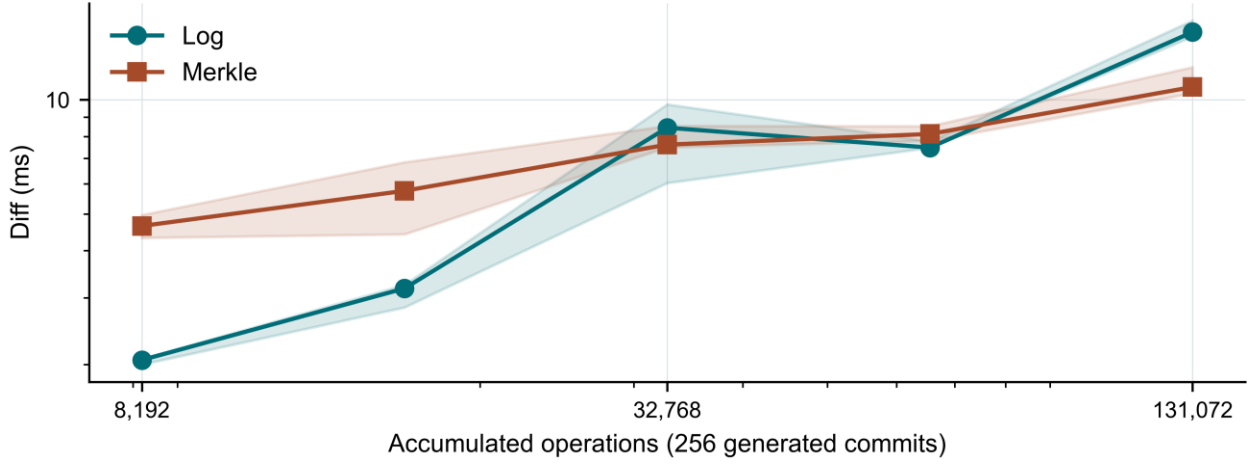


Fig. 5. Exploratory long-history diagnostic, 10k rows and 256 repeated-key commits; medians and IQRs. This extension does not alter the frozen threshold. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

The diagnostic median log/Merkle ratios are 0.44 at 8,192, 0.55 at 16,384, 1.11 at 32,768, 0.92 at 65,536, 1.40 at 131,072 operations. Merkle first has the lower sampled median at 32,768, log is lower again at 65,536, and Merkle is lower at 131,072. This non-monotonic ordering is not a resolved single crossover or an optimal switching rule; IQRs and raw queries are retained, and only one seed was used.

requested diff strategy changes. Table VI reports all eight primary workloads; Table VII reports the separate public-record sweep. Figure 6 displays medians and IQRs. These measurements isolate diff-path selection more directly than cross-system workflows, although run order, runtime effects and the common output contract still affect observed latency.

Figure 6 compares both variants across synthetic and public-record workloads.

V. RESULTS

A. Primary hybrid ablation

Revon-M versus Revon-H is the central ablation: both use the same persistent trie and addressed changesets, and the

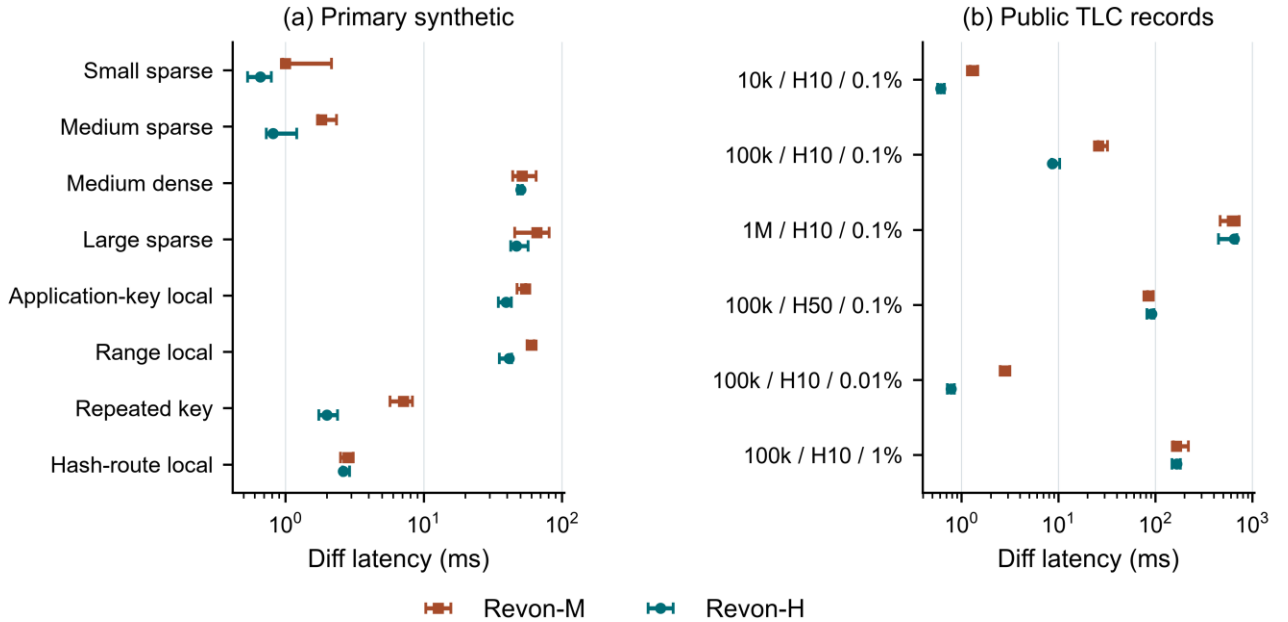


Fig. 6. Revon-M and Revon-H ablation; primary synthetic workloads and public TLC workloads are separate panels. Points show medians and bars show IQRs. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

TABLE VI. Primary ablation; paired ratio intervals on fixed synthetic workloads

Workload	M ms [IQR]	H ms [IQR]	M/H [95% CI]	n
Small sparse	1.00 [0.96, 2.15]	0.66 [0.53, 0.79]	2.09 [1.29, 3.77]	7
Medium sparse	1.83 [1.73, 2.35]	0.81 [0.73, 1.21]	1.97 [1.51, 3.51]	7
Medium dense	51.36 [44.08, 65.12]	50.39 [47.95, 51.74]	1.00 [0.93, 1.51]	7
Large sparse	65.73 [45.51, 80.93]	46.83 [42.69, 56.85]	1.29 [0.95, 1.55]	7
Application-key local	54.40 [47.29, 54.68]	39.36 [34.66, 43.04]	1.39 [1.06, 1.74]	7
Range local	60.41 [55.94, 62.97]	41.36 [35.21, 43.26]	1.47 [1.28, 1.80]	7
Repeated key	7.15 [5.72, 8.27]	1.99 [1.74, 2.38]	3.52 [2.66, 4.15]	7
Hash-route local	2.83 [2.51, 3.09]	2.62 [2.59, 2.91]	1.09 [0.95, 1.17]	7

A ratio above one favours Revon-H; intervals containing one are inconclusive. Three primary intervals contain parity, including the dense case in which both variants use Merkle comparison. The separate three-seed 50-commit result is

0.99 [0.90, 1.17] across 21 paired trials and also remains inconclusive. Four other robustness factors favour Revon-H on both Windows and WSL2, as documented in their separate seed-cluster analyses.

TABLE VII. Public-record ablation; exploratory unpaired ratio-of-medians intervals on fixed workloads

Workload	M ms [IQR]	H ms [IQR]	M/H [95% CI]	n
10k / H10 / 0.1%	1.27 [1.23, 1.47]	0.61 [0.57, 0.67]	2.07 [1.83, 2.63]	7
100k / H10 / 0.1%	25.95 [25.74, 32.02]	8.65 [8.54, 10.28]	3.00 [2.27, 3.99]	7
1M / H10 / 0.1%	621.49 [466.41, 731.30]	651.01 [448.66, 693.13]	0.95 [0.63, 1.85]	7
100k / H50 / 0.1%	83.68 [83.54, 95.00]	91.79 [82.20, 97.15]	0.91 [0.83, 1.17]	7
100k / H10 / 0.01%	2.78 [2.50, 3.16]	0.78 [0.72, 0.85]	3.57 [2.85, 4.47]	7
100k / H10 / 1%	164.79 [153.96, 218.70]	166.75 [149.23, 181.91]	0.99 [0.81, 1.48]	7

The public 50-commit case has an M/H ratio of 0.91 [0.83, 1.17] and median latencies 83.68 ms for M and 91.79 ms for H. Its point estimate favours M. This is a separate update-only workload and must not be presented as resolving the earlier mixed-workload 50-commit result. The million-

record recovery was outside the original block, which further limits causal interpretation of that configuration.

B. Complete workflow comparison with Dolt

Figure 7 compares the complete Revon-H and Dolt diff workflows.

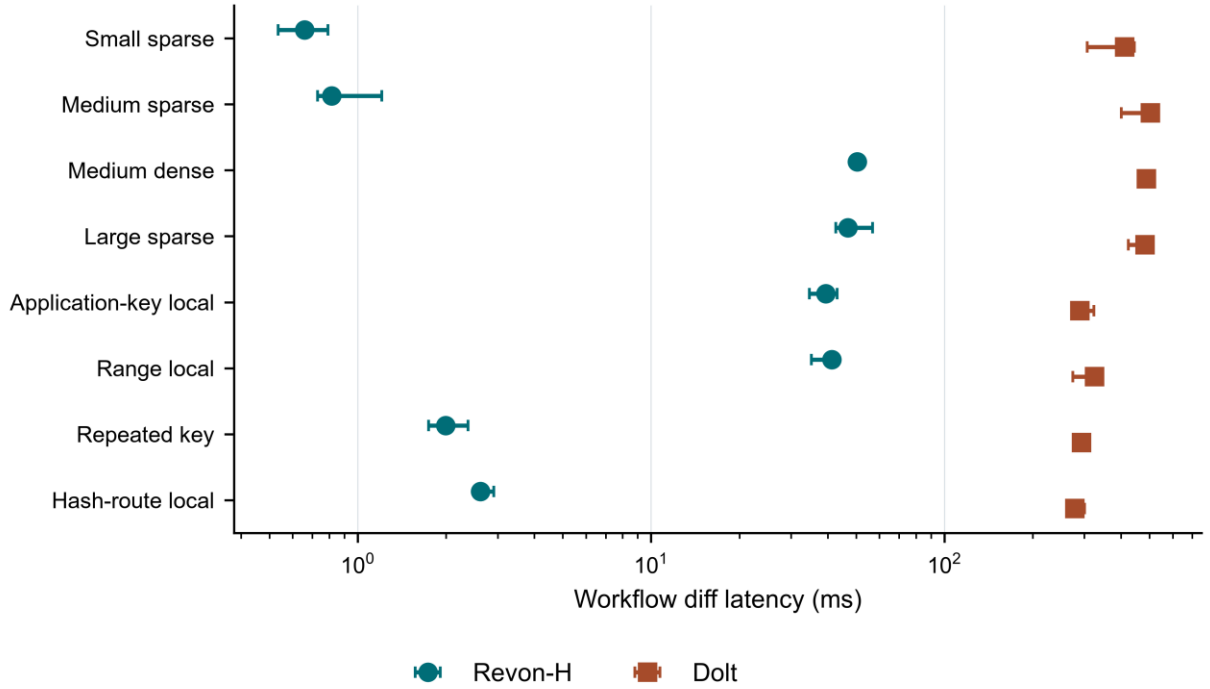


Fig. 7. Revon-H API and Dolt SQL/CLI diff workflows; medians and IQRs across eight synthetic workloads. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

Table VIII reports the paired workflow comparison with Dolt.

TABLE VIII. Revon-H and SQL-path Dolt diff latency; median paired ratios and 95% bootstrap intervals

Scenario	Diff path	Revon-H (ms)	Dolt SQL (ms)	Dolt/Revon-H ratio (95% CI)
Small sparse	log	0.659	411.69	625.1 (537.1-658.7)
Medium sparse	log	0.814	503.31	482.5 (393.4-652.0)
Medium dense	merkle	50.391	488.42	10.3 (9.0-10.8)
Large sparse	log	46.834	483.08	10.8 (8.2-12.4)
Application-key local	log	39.361	289.87	7.1 (6.8-9.8)
Range local	log	41.359	325.04	7.8 (6.8-11.2)
Repeated key	log	1.992	293.72	141.6 (109.5-173.2)
Hash-route local	log	2.620	278.22	102.4 (97.5-104.0)

In all eight scenarios, the paired Dolt/Revon-H diff ratio exceeded one and its 95% interval excluded parity. Revon-H therefore had lower diff latency in the workflows we tested. Table VIII gives the median of the seven same-trial ratios and the bootstrap interval. The comparison is between Revon’s in-process API and Dolt’s CLI workflow, which includes Dolt CLI and SQL costs; these ratios are not engine-only speedups. The lexically adjacent application-key workload is distinct from the range-local, repeated-key, and hash-route-local workloads.

C. Latency and storage attribution

Diff-path selection does not imply a second persistent representation. Both M and H retain addressed changesets and trie objects. Their measured footprints are nearly equal. The large Revon-versus-Dolt footprint difference therefore cannot be assigned to hybrid selection. Table IX reports the actual M/H storage difference and point-estimate latency reduction; a negative reduction means H is slower. Percentage reductions are computed from scenario medians and are distinct from the paired median-ratio statistic in Table VI.

TABLE IX. Latency and storage differences in primary synthetic workloads; directory footprints before compaction

Workload	M MiB	H MiB	Storage change %	H latency reduction %
Small sparse	0.71	0.71	+0.000	+34.2
Medium sparse	2.91	2.91	+0.000	+55.5
Medium dense	10.66	10.68	+0.183	+1.9
Large sparse	34.44	34.42	-0.057	+28.7
Application-key local	34.44	34.43	-0.034	+27.6
Range local	34.43	34.42	-0.045	+31.5
Repeated key	34.48	34.48	+0.000	+72.1
Hash-route local	29.29	29.28	-0.027	+7.4

Figure 8 places measured diff latency against repository footprint for each variant.

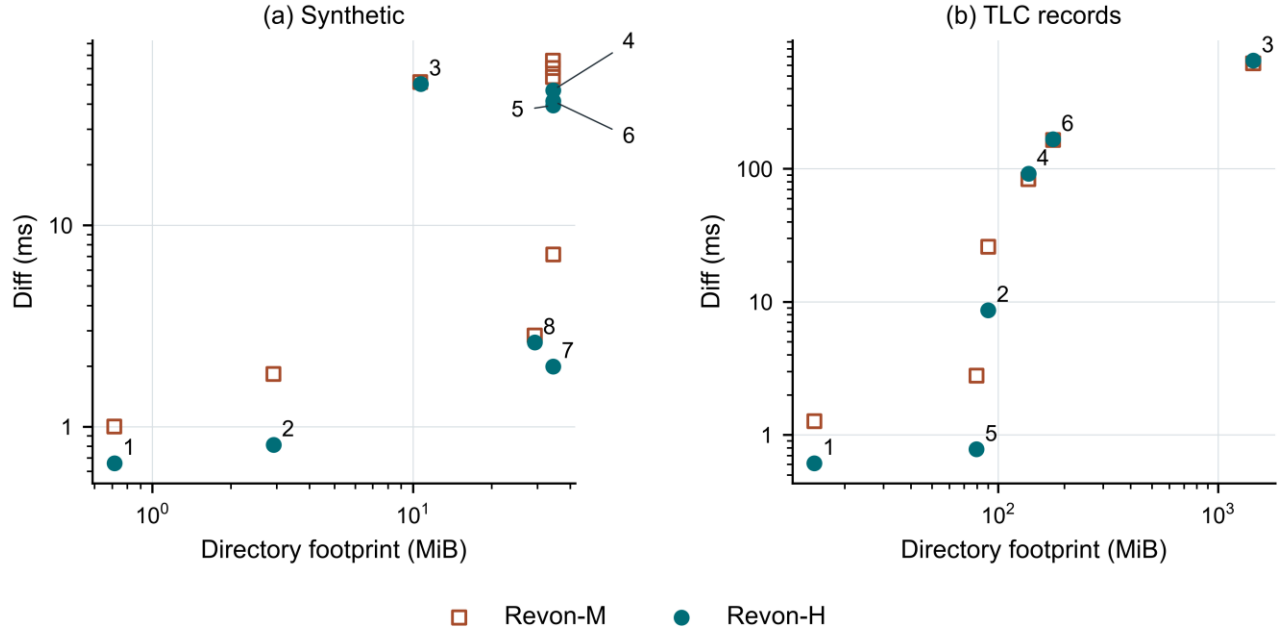


Fig. 8. Latency versus directory footprint for M and H; panels use the row order of Tables VI and VII. Numerals identify H points, and overlapping footprints do not imply missing M measurements. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

Repository directory size measures the footprint of the full workflow, not storage normalized to the algorithms alone. Across the eight scenarios, Revon-H’s repository footprint was 1.99-6.90 times Dolt’s. After dividing each directory size by the final UTF-8 key/value payload, the median Revon-H/Dolt ratio was 3.82; the corresponding ratio for the residual was 6.82. That residual is not pure metadata: it also includes serialization, indexes, other metadata, and compression effects. After compaction, mean sampled footprints were 22.08 MiB for Revon-M, 22.08 MiB for Revon-H, 2.17 MiB for Dolt SQL, and 2.19 MiB for Dolt bulk import. We compacted Dolt offline with ‘dolt gc’ and

the Revon SQLite repositories with ‘VACUUM’; compaction time is excluded from latency and RSS. These are sampled outcomes, not a pure metadata comparison or a universal steady-state result.

The supplemental repositories measure canonical object payload bytes by kind before deletion of the reproducible working database. Node and changeset payload totals are exact SQL sums, not complete allocated storage costs. For the held-out 100k-row, 64-commit spread workload at 32,768 operations, the closed file is 188.59 MiB: canonical node payloads total 60.34 MiB, changesets 15.21 MiB and

commits 0.02 MiB. The remaining 113.02 MiB includes hashes, indexes, metadata, page structure and free space; it is not pure metadata overhead. This is a supplemental repository measurement, separate from the primary workflow directory footprints. These measurements identify retained object categories but do not measure the footprint of a changeset-free variant. Such a variant is not needed to

compare existing M and H, and no equivalent variant is claimed here.

D. Commit checkout and trie geometry

Figure 9 reports commit and checkout workflow latency separately from diff latency.

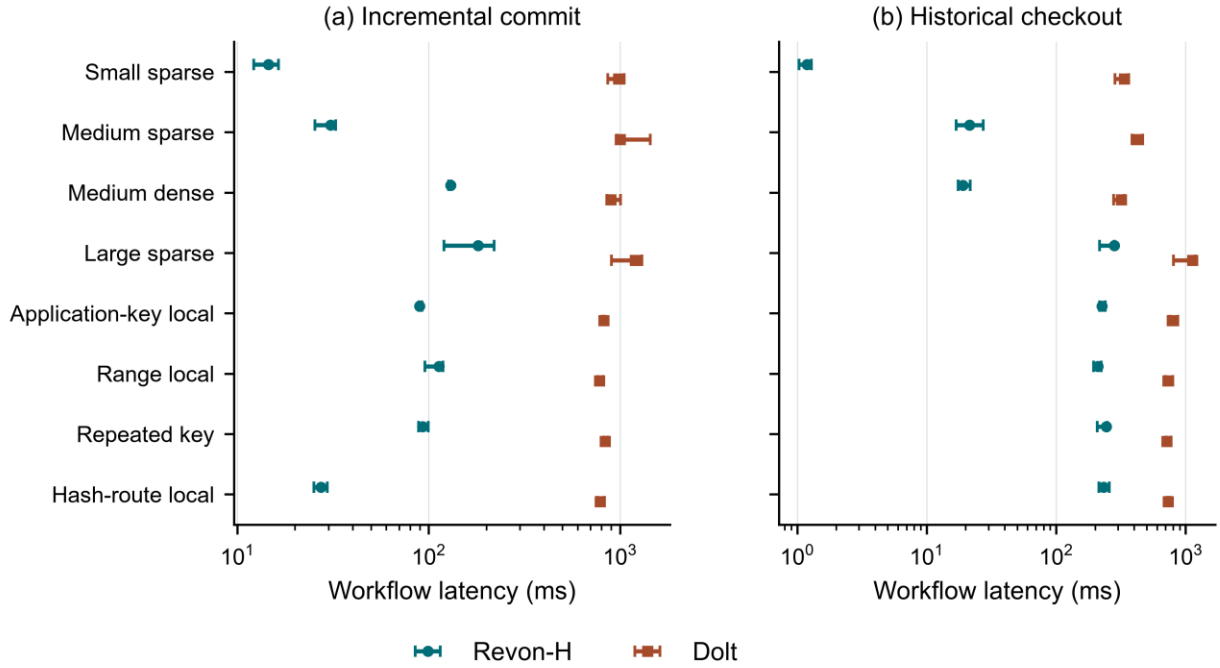


Fig. 9. Incremental-commit and historical-checkout workflow latency; medians and IQRs for Revon-H and Dolt. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

Across all eight scenarios, the paired Dolt/Revon-H intervals excluded parity for initial import, incremental commit, diff, and historical checkout. Full ratios and intervals appear in `paired_ratio_uncertainty.csv`. Median initial-import times across scenario medians were 1,814.83 ms for batched SQL INSERT, 1,701.25 ms for Dolt bulk import, and 1,131.24 ms for Revon-H. In the paired SQL-versus-bulk-import

analysis, four scenario intervals favored bulk import and four included parity; the results do not show a general bulk-import advantage. These are workflow measurements, not evidence of engine-level or production-system superiority.

Figure 10 examines the measured latency and footprint of alternative fixed geometries.

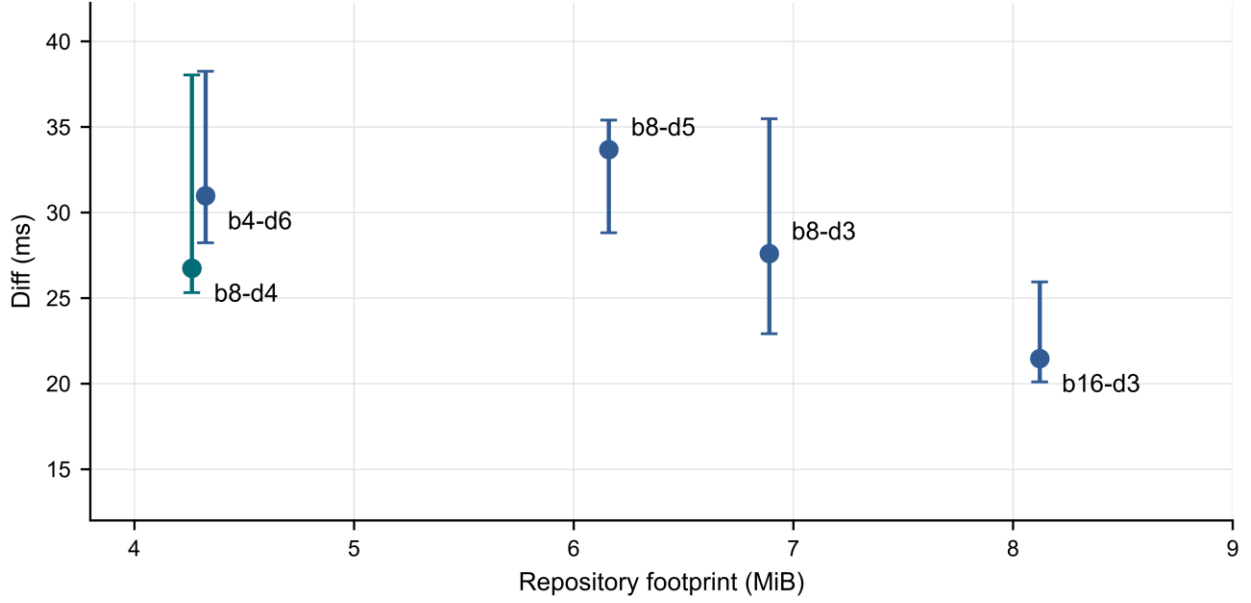


Fig. 10. Fixed-trie geometry sensitivity; points show median diff latency and bars show IQRs for seven measured trials. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

The corrected fixed-trie sensitivity run completed 45 executions: 10 warm-ups and 35 measured runs. The b16-d3 configuration had the lowest median diff time (21.46 ms), but used 8.12 MiB. The b8-d4 configuration took 26.74 ms and used the least storage (4.26 MiB). The over-deep b8-d5 configuration took 33.68 ms to diff and used 44.5% more

storage than b8-d4. Thus, b8-d4 balances latency and storage in this sample; it is not a universal optimum.

E. Public dataset scale history and update density

Figure 11 reports the separate public-record scalability sweep.

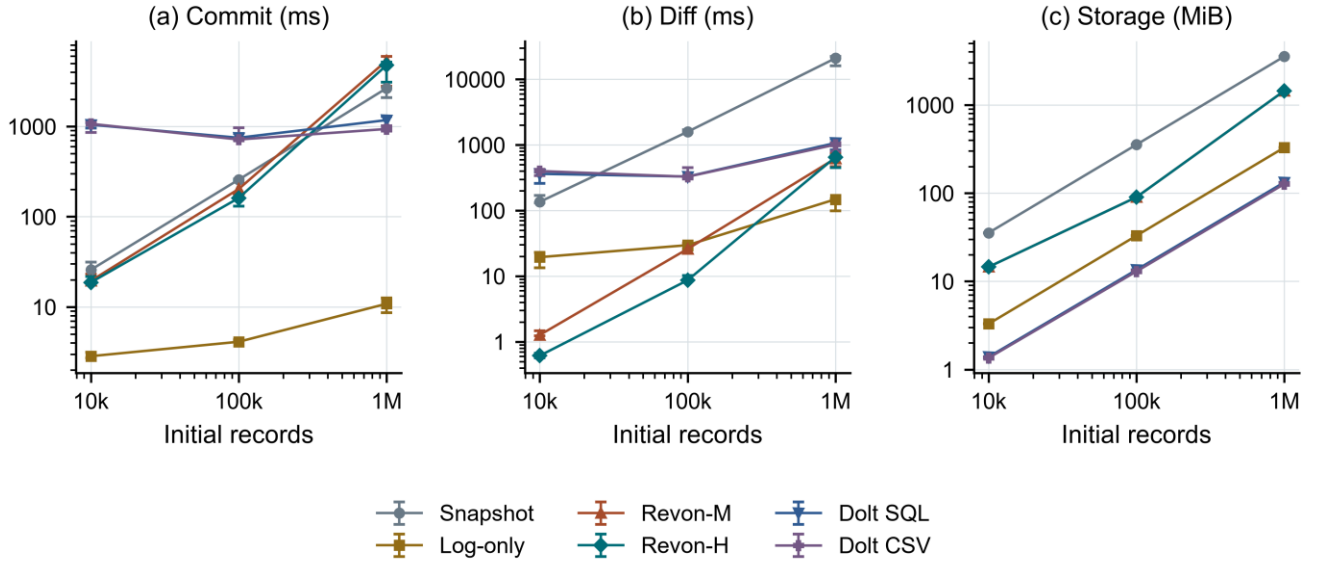


Fig. 11. Public-record scalability for ten generated commits and 0.1% updates per commit; medians and IQRs from seven successful measured trials. Both axes are logarithmic. H uses log diff at 10k/100k and Merkle diff at 1M. Plotting/diagram code assisted by OpenAI Codex; content derives from measured data or inspected code.

At one million records, Revon-H's median incremental commit was 4764.26 [3086.15, 5159.49] ms and its initial-to-final diff was 651.01 [448.66, 693.13] ms. Its median repository footprint was 1446.69 MiB. Both Revon variants selected Merkle differencing for the 10,000 accumulated

operations at one million records; this configuration does not test a log-path benefit. Revon-H's incremental-commit median exceeded those of Snapshot, Log-only, Dolt SQL, Dolt CSV at this scale.

Initial import for Revon-H took 39.60 [27.54, 41.41] s, compared with 29.02 [20.93, 37.70] s for Dolt SQL and 24.94 [20.11, 27.29] s for Dolt CSV import. These initial loads are separate from incremental commits. The ratio of median Revon-H/Dolt SQL repository footprints was 10.90; this includes serialization, indexes and compression effects and is not a pure metadata comparison.

Brackets give the interquartile range across measured trial summaries, not confidence intervals or variation across datasets. Full import, checkout, storage and failure records remain in the evidence bundle.

Failed executions are retained and excluded from performance summaries. One interrupted million-record Revon-M attempt was followed by a fresh recovery trial after the matrix, under identical frozen source and workload. Seven successful measured trials enter each summary; the failed attempt remains in the original bundle. The recovery was outside the counterbalanced block and host memory conditions were not held constant.

Table X reports public-record history and density results at 100,000 records.

TABLE X. Public history and density results at 100k records; median commit and diff times in ms and footprint in MiB

H / density	System	Commit	Diff	MiB
10 / 0.1%	Snapshot	257.41	1584.12	354.91
10 / 0.1%	Log-only	4.11	29.62	32.93
10 / 0.1%	Revon-M	202.78	25.95	90.03
10 / 0.1%	Revon-H	161.09	8.65	90.04
10 / 0.1%	Dolt SQL	749.08	329.10	13.50
10 / 0.1%	Dolt CSV	716.78	327.17	12.95
50 / 0.1%	Snapshot	219.28	1377.48	1645.60
50 / 0.1%	Log-only	3.26	52.11	35.58
50 / 0.1%	Revon-M	135.36	83.68	137.24
50 / 0.1%	Revon-H	141.22	91.79	137.24
50 / 0.1%	Dolt SQL	882.14	590.91	25.88
50 / 0.1%	Dolt CSV	873.70	525.45	25.33
10 / 0.01%	Snapshot	236.94	1257.26	354.90
10 / 0.01%	Log-only	3.00	21.38	32.33
10 / 0.01%	Revon-M	26.57	2.78	79.68
10 / 0.01%	Revon-H	28.56	0.78	79.68
10 / 0.01%	Dolt SQL	1001.47	374.25	10.85
10 / 0.01%	Dolt CSV	1015.52	367.27	10.30
10 / 1%	Snapshot	265.73	1738.57	354.97
10 / 1%	Log-only	11.04	146.20	38.90
10 / 1%	Revon-M	752.02	164.79	177.68
10 / 1%	Revon-H	679.38	166.75	177.66

H / density	System	Commit	Diff	MiB
10 / 1%	Dolt SQL	943.05	740.59	25.31
10 / 1%	Dolt CSV	976.99	702.91	24.75

At 100,000 records, the 10-commit 0.1% case accumulates 1,000 operations, while the 50-commit case accumulates 5,000 and the 1% case accumulates 10,000. Revon-H therefore uses the log path for the sparse short history and the Merkle path for the latter cases. This sweep evaluates the frozen selector under public record contents; it does not establish a new threshold or universal advantage. The update-only histories differ from the earlier mixed synthetic workloads.

VI. DISCUSSION

For RQ1, the public-record study shows a material scale cost: at one million records, H's 4,764.26 ms commit median exceeds the other measured workflows, and its 1,446.69 MiB footprint is 10.90 times Dolt SQL's. Lower diff latency in selected workflows does not establish general efficiency. Interface-level Dolt ratios do not establish an intrinsically faster trie.

For RQ2, adaptive differencing helps selected sparse and repeated-key workloads without changing their version semantics. M/H footprints are nearly equal because the representations are shared. Dense and long histories do not consistently favour H. The new threshold supplement shows that 4,096 is a conservative frozen rule for earlier results, not a measured universal crossover; 16,384 is the best sampled candidate in a distinct cached-query protocol.

For RQ3, the sampled geometries trade latency against storage; no universal optimum is established. Shallow routing reduces internal traversal but leaves larger buckets and higher storage use. Deeper routing reduces bucket occupancy but adds internal-node work. The suitable configuration depends on whether a workload prioritizes latency, storage, or limiting leaf scans.

The Dolt ratios reflect the complete systems and interfaces we measured. Dolt handles SQL parsing, schema management, version-control operations, and production durability, which the Revon prototype does not. Its Prolly Trees also support ordered queries, branches, merges, and remote workflows. Because our timings include each system's declared interface, they do not isolate the tree algorithms. We therefore cannot conclude that a fixed trie is inherently faster than a Prolly Tree.

The public-data sweeps extend the evidence to real record contents and larger states, but the histories remain generated. At fixed depth, increasing N increases expected leaf occupancy, so rewriting touched leaves can become more expensive even with an unchanged update percentage. The scale sweep also crosses the frozen selector threshold: Revon-H uses log diff at 10k/100k records and Merkle diff at 1M. Its diff curve therefore combines larger states with a

strategy change. Longer histories likewise accumulate operations and can move Revon-H to the Merkle path. Figure 11 and Table X describe this host and record mapping; they do not isolate a tree algorithm, test relational queries, or measure naturally occurring corrections.

VII. LIMITATIONS AND THREATS TO VALIDITY

Runtime and interfaces: Revon runs in-process in Python, while Dolt is a Go executable reached through CLI/SQL. The measured workflows differ in schema and feature costs. Revon supports one writer and linear history; SQL, branching, merging, remotes, schema evolution, ordered range scans and concurrent workloads are not evaluated.

Dataset scope: The earlier synthetic study reaches 100,000 rows; the new public-record evaluation completes configurations up to 1,000,000 rows from one TLC monthly file with eleven retained fields. All public-data updates are generated, the scale sample uses one seed and an affine permutation, and key identities are valid only within the frozen release. We did not evaluate observed histories, multiple public datasets, multi-table workloads, relational queries or scale beyond the completed configurations.

Environment and replication: Windows and WSL2 measurements use the same physical computer. No independent Linux machine was available to this revision, and no external researcher or clean clone has reproduced the timings. The internal audit checks evidence consistency; it does not reproduce wall-clock performance. Broader hardware claims remain unsupported.

Calibration: the five-candidate, predefined held-out supplement studies cached queries on synthetic workloads with one seed per profile and one repository per operation count. The largest tested candidate wins, so the search does not establish an optimum. The long-history extension is exploratory and excluded from selection. Query repetitions do not supply independent dataset or repository replications. The fixed-depth trie does not adapt its geometry.

Memory: We measured median whole-worker process-tree RSS in eight primary scenarios for Snapshot, Log-only, Revon-M, Revon-H, and SQL-path Dolt. These measurements include shared workload/oracle state and adapter processes, so they are not product-only memory figures. The parent process sampled every 10 ms outside timed operations. It may miss shorter peaks, and monitoring may add small scheduling pressure. Dolt bulk-import measurements are reported separately from SQL-path Dolt. Public-data trials also include workload/oracle memory. The host had limited available RAM at launch; memory pressure and paging costs were not isolated from the workflow, so the

measured scaling should not be read as an algorithm-only complexity result.

Work units: Keys, log operations, trie-node pairs, and Dolt’s internal work are different quantities. We do not combine them into a normalized work measure.

Uncertainty: We retained every Tukey outlier candidate. The primary seven-trial ratio intervals are paired bootstrap intervals for each fixed scenario. The additional seed-cluster bootstrap uses only three seeds. Neither analysis estimates performance across a population of datasets or machines.

Storage and scope: both M and H retain changesets. Canonical payload accounting excludes allocated index and page costs, and comparing roots does not itself explain the repository footprint. Here, adaptive refers solely to diff-path selection; the geometry is fixed. The results cover selected versioning operations on a prototype and do not establish general efficiency across structured-data management.

VIII. FUTURE WORK

Priority work is independent hardware replication and additional public datasets with observed histories. Threshold calibration should test a wider candidate range, multiple seeds per profile, different output sizes, and cold-storage behaviour before deployment. A future selector could use measured prefix density and I/O as well as operation count; none of that richer classification exists in the present implementation.

Revon’s storage could be reduced through binary node records, separate value chunks, packed objects, compression, reachability checks, and garbage collection. An ordered Merkle index or content-defined chunking could add range scans and history-independent boundaries while retaining the fixed trie as a controlled baseline.

Named branches, merge commits, conflict reporting, schema-aware records, concurrency and ordered queries require implementation and separate correctness/performance studies. They are future capabilities rather than demonstrated properties.

IX. CONCLUSION

Revon combines immutable objects, incremental trie updates, SQLite persistence, historical checkout and adaptive log/Merkle differencing. A prominent same-representation ablation supports advantages for selected sparse and repeated-key intervals, with dense and long-history qualifications retained. Public records reach one million rows using generated corrections and expose substantial commit and storage costs. New calibration and held-out measurements improve threshold evidence without rewriting earlier results or establishing a universal optimum. The contribution is a reproducible comparison of diff choices in a single-writer prototype; independent hardware validation remains uncompleted.

APPENDIX A. REPRODUCIBILITY AND DATA AVAILABILITY

The primary synthetic evidence is evidence/paper-corrected-issues13-17-counterbalanced-final-20260923, with paired_ratio_uncertainty.csv. Separate robustness, WSL and telemetry bundles are evidence/paper-issues18-20-robustness-20260923, evidence/paper-issues18-20-linux-wsl-20260923 and evidence/paper-issues14-windows-telemetry-20260924. Trie sensitivity is evidence/trie-sensitivity-issues8-20260923. Public TLC analysis is evidence/public-tlc-combined-20261002, referring to its original execution and isolated recovery parents. Pilots are excluded. Local revision evidence is not yet a public release.

New held-out threshold evidence is evidence/threshold-heldout-20261002. Reproduce with Python 3.14: `python -m experiments.threshold_validation --output <new-directory>`. The diagnostic extension is evidence/crossover-diagnostic-20261002, reproduced with `python -m experiments.crossover_extension --output <new-directory>`. Each bundle archives exact source files and hashes, workloads, raw query records, selection, summaries and correctness checks. The pilot at evidence/threshold-pilot-20261002 is excluded. Working databases are reproducible and removed after verified reopen checks; payload accounting and integrity counts are retained.

DECLARATIONS

Funding: the funding statement must be confirmed by the authors before submission.

Competing interests and author contributions: author-confirmed declarations have not yet been supplied for this revision. Names, affiliations and contact details are preserved from the reviewed manuscript.

Ethics: this revision uses public TLC trip records and generated modifications. Any required ethics or exemption statement must be confirmed by the authors for the journal’s policy; no approval is invented.

AI assistance: OpenAI Codex assisted code development, analysis scripting, manuscript restructuring and editing, and plotting code during this revision. Graphs are generated from retained measured outputs or the inspected implementation, not invented empirical data. The authors must review the revised arguments, interpretations, references and artifacts and confirm the final disclosure and accountability before submission.

Data and code availability: the existing project repository is <https://github.com/atharvasheersh/Revon>. TLC acquisition and checksum-pinned preparation are documented in docs/experiments/PUBLIC_DATASET_EVALUATION.md [31]. The new public-data and threshold evidence is retained locally with this revision; public release of these new bundles is pending. Appendix A identifies the exact bundles and reproduction commands. Source trip files are downloaded separately and excluded from Git.

ACKNOWLEDGMENT

We thank the open-source Dolt project and the researchers whose work on dataset versioning informed our experimental design.

REFERENCES

- [1] S. Chacon and B. Straub, “Git Internals: Git Objects,” in *Pro Git*, 2nd ed., 2014. [Online]. Available: <https://git-scm.com/book/en/v2/Git-Internals-Git-Objects>.
- [2] DoltHub, “Dolt Storage Engine: Block Store,” Dolt Documentation, accessed Oct. 2, 2026. [Online]. Available: <https://www.dolthub.com/docs/architecture/storage-engine/block-store/>.
- [3] T. Sehn, “Dolt’s Storage Engine,” DoltHub Blog, Feb. 29, 2024. [Online]. Available: <https://www.dolthub.com/blog/2024-02-29-storage-engine/>.
- [4] R. C. Merkle, “A digital signature based on a conventional encryption function,” in *Advances in Cryptology: CRYPTO ’87, Lecture Notes in Computer Science*, vol. 293, 1988, pp. 369-378, https://doi.org/10.1007/3-540-48184-2_32. Open copy: <https://people.eecs.berkeley.edu/~raluca/cs261-f15/readings/merkle.pdf>.
- [5] J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan, “Making data structures persistent,” *J. Comput. Syst. Sci.*, vol. 38, no. 1, pp. 86-124, 1989, [https://doi.org/10.1016/0022-0000\(89\)90034-2](https://doi.org/10.1016/0022-0000(89)90034-2). Open copy: <https://www.cs.cmu.edu/~sleator/papers/making-data-structures-persistent.pdf>.
- [6] J. Benet, “IPFS: Content Addressed, Versioned, P2P File System,” *arXiv:1407.3561*, 2014. [Online]. Available: <https://arxiv.org/abs/1407.3561>.
- [7] A. Auvolat and F. Taïani, “Merkle Search Trees: Efficient State-Based CRDTs in Open Networks,” in *Proc. 38th IEEE Int. Symp. Reliable Distributed Syst. (SRDS)*, 2019, pp. 221-230, <https://doi.org/10.1109/SRDS47363.2019.00032>. Open copy: <https://zenodo.org/records/3935655/files/article.pdf>.
- [8] A. Muthitacharoen, B. Chen, and D. Mazières, “A low-bandwidth network file system,” in *Proc. 18th ACM Symp. Operating Syst. Principles (SOSP)*, 2001, pp. 174-187, <https://doi.org/10.1145/502034.502052>. Open copy: <https://pdos.csail.mit.edu/papers/lbfs:sosp01/lbfs.pdf>.
- [9] A. Bhardwaj et al., “DataHub: Collaborative Data Science and Dataset Version Management at Scale,” in *Proc. 7th Biennial Conf. Innovative Data Syst. Research (CIDR)*, 2015. Open copy: https://www.cidrdb.org/cidr2015/Papers/CIDR15_Paper18.pdf.
- [10] M. Maddox et al., “Decibel: The Relational Dataset Branching System,” *Proc. VLDB Endowment*, vol. 9, no. 9, pp. 624-635, 2016, <https://doi.org/10.14778/2947618.2947619>. Open copy: <https://www.vldb.org/pvldb/vol9/p624-maddox.pdf>.
- [11] S. Huang, L. Xu, J. Liu, A. J. Elmore, and A. Parameswaran, “OrpheusDB: Bolt-on Versioning for Relational Databases,” *Proc. VLDB Endowment*, vol. 10, no. 10, pp. 1130-1141, 2017, <https://doi.org/10.14778/3115404.3115417>. Open copy: <https://www.vldb.org/pvldb/vol10/p1130-huang.pdf>.
- [12] S. Bhattacharjee, A. Chavan, S. Huang, A. Deshpande, and A. Parameswaran, “Principles of Dataset Versioning: Exploring the Recreation/Storage Tradeoff,” *Proc. VLDB Endowment*, vol. 8, no. 12, pp. 1346-1357, 2015, <https://doi.org/10.14778/2824032.2824035>. Open copy: <https://www.vldb.org/pvldb/vol8/p1346-bhattacharjee.pdf>.
- [13] M. Armbrust et al., “Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores,” *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3411-3424, 2020, <https://doi.org/10.14778/3415478.3415560>. Open copy: <https://www.vldb.org/pvldb/vol13/p3411-armbrust.pdf>.
- [14] Attic Labs, “Noms Technical Overview.” GitHub. [Online]. Available: <https://github.com/attic-labs/noms/blob/master/doc/intro.md>.
- [15] S. Wang et al., “ForkBase: An Efficient Storage Engine for Blockchain and Forkable Applications,” *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1137-1150, 2018, <https://doi.org/10.14778/3231751.3231762>. Open copy: <https://www.vldb.org/pvldb/vol11/p1137-wang.pdf>.
- [16] A. Twigg, A. Hyde, G. Miłos, T. Moreton, J. Wilkes, and T. Wilkie, “Stratified B-trees and versioning dictionaries,” in *Proc. 3rd USENIX Workshop on Hot Topics in Storage and File Systems (HotStorage)*, 2011. [Online]. Available: <https://arxiv.org/abs/1103.4282>.
- [17] A. Seering, P. Cudré-Mauroux, S. Madden, and M. Stonebraker, “Efficient versioning for scientific array databases,” in *Proc. IEEE 28th Int. Conf. Data Engineering (ICDE)*, 2012, pp. 1013-1024, <https://doi.org/10.1109/ICDE.2012.102>. [Online]. Available: <https://dspace.mit.edu/entities/publication/a5c2c050-6abd-47c1-8bee-4f8c51b6a15f>.
- [18] E. Soroush and M. Balazinska, “Time travel in a scientific array database,” in *Proc. IEEE 29th Int. Conf. Data Engineering (ICDE)*, 2013, pp. 98-109, <https://doi.org/10.1109/ICDE.2013.6544817>. [Online]. Available: https://scidb.cs.washington.edu/paper/ICDE13_conf_full_422.pdf.
- [19] G. S. Brodal, S. Sioutas, K. Tsakalidis, and K. Tsichlas, “Fully persistent B-trees,” *Theoretical Computer Science*, vol. 841, pp. 10-26, 2020, <https://doi.org/10.1016/j.tcs.2020.06.027>. [Online]. Available: <https://cs.au.dk/~gerth/papers/tcs20.pdf>.
- [20] N. Saquib, C. Krintz, and R. Wolski, “Replicated versioned data structures for wide-area distributed systems,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 1, pp. 207-224, 2023, <https://doi.org/10.1109/TPDS.2022.3217969>. [Online]. Available: <https://racelab.cs.ucsb.edu/papers/tpds22-versioned.pdf>.
- [21] M. E. Schüle, L. Karnowski, J. Schmeißer, B. Kleiner, A. Kemper, and T. Neumann, “Versioning in main-memory database systems: From MusaeusDB to TardisDB,” in *Proc. 31st Int. Conf. Scientific and Statistical Database Management (SSDBM)*, 2019, pp. 169-180, <https://doi.org/10.1145/3335783.3335792>. [Online]. Available: <https://www-db.cs.tum.edu/~schuele/data/tardis.pdf>.

- [22] P. Buneman, S. Khanna, and W.-C. Tan, "Why and where: A characterization of data provenance," in Database Theory: ICDT 2001, Lecture Notes in Computer Science, vol. 1973, 2001, pp. 316-330, https://doi.org/10.1007/3-540-44503-X_20. [Online]. Available: https://www.pure.ed.ac.uk/ws/files/16509989/Why_and_Where_A_Characterization_of_Data_Provenance.pdf.
- [23] F. McSherry, D. G. Murray, R. Isaacs, and M. Isard, "Differential dataflow," in Proc. 6th Biennial Conf. Innovative Data Systems Research (CIDR), 2013. [Online]. Available: https://www.cidrdb.org/cidr2013/Papers/CIDR13_Paper111.pdf.
- [24] M. Akhlaghi et al., "Toward long-term and archivable reproducibility," Computing in Science & Engineering, vol. 23, no. 3, pp. 82-91, 2021, <https://doi.org/10.1109/MCSE.2021.3072860>. [Online]. Available: <https://arxiv.org/abs/2006.03018>.
- [25] R. Shraga and R. J. Miller, "Explaining dataset changes for semantic data versioning with Explain-Da-V," arXiv:2301.13095, 2023, technical report/preprint. [Online]. Available: <https://arxiv.org/abs/2301.13095>.
- [26] A. Guo, J. Li, P. Sukprasert, S. Khuller, A. Deshpande, and K. Mukherjee, "To store or not to store: A graph theoretical approach for dataset versioning," in Proc. IEEE Int. Parallel and Distributed Processing Symposium (IPDPS), 2024, pp. 479-493, <https://doi.org/10.1109/IPDPS57955.2024.00049>. [Online]. Available: <https://arxiv.org/abs/2402.11741>.
- [27] A. González-Cebrián, M. Bradford, A. E. Chis, and H. González-Vélez, "Standardised versioning of datasets: A FAIR-compliant proposal," Scientific Data, vol. 11, art. 358, 2024, <https://doi.org/10.1038/s41597-024-03153-y>. [Online]. Available: <https://www.nature.com/articles/s41597-024-03153-y>.
- [28] J. Zhao and S. Krishnan, "Compression and in-situ query processing for fine-grained array lineage," arXiv:2405.17701, 2024. [Online]. Available: <https://arxiv.org/abs/2405.17701>.
- [29] H. Gou, Z. Zhang, Y. Sun, P. Xu, F. Tian, L. Wang, and J. Wang, "Git4Data: Database-native version control for AI agents," arXiv:2609.02106, 2026, preprint. [Online]. Available: <https://arxiv.org/abs/2609.02106>.
- [30] J. Edwards and T. Petricek, "Baseline: Operation-based evolution and versioning of data," arXiv:2512.09762, 2025, preprint, updated 2026. [Online]. Available: <https://arxiv.org/abs/2512.09762>.
- [31] New York City Taxi and Limousine Commission, "TLC Trip Record Data: Yellow Taxi, January 2024," public dataset, accessed Oct. 1, 2026. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>.